

Booting Windows from USB – Michael Waterman

 michaelwaterman.nl/2022/12/23/booting-windows-from-usb

Michael Waterman

December 23, 2022



I sometimes get nostalgic about the technology that was hot and happening a few years ago, this is especially true for Windows To Go. A nice little piece of technology that allows Windows to boot from a specially crafted USB stick. When it was introduced, it required specific USB pen drives as, at the time, USB equipped with internal fast storage was not really that common. Compared to what's available now it was very expensive and limited to a few vendors. Although the UI for creating a WTG pen drives has been removed it's really not that hard to manually create one.

As I'm mostly security focused, you could argue, why this post? Well, I see a lot of admins still struggling with the concept of "Secure Admin Workstations", especially in the MSP space. The usual remark that I get is that they have to carry around multiple laptops to securely manage the infrastructure. Now, I'm not saying that this is THE solution, but perhaps it can fit in a number of scenarios that you may encounter. At least you can save on additional hardware cost and not having to carry around an abundance of hardware. Let's dive in...

Note! You will be needing a fast USB 3 storage device that is defined as "internal" storage, not as removable. Most cheaper USB storage devices are identified as removable, so be careful what you buy. For this demo I'm using an "Intenso Portable SSD Premium 128GB" disk.

Configuring the disk

Start PowerShell as administrator and following along, I'll explain every command and what we are doing along the way. We'll start at the basic level first, detecting and configuring the disk.

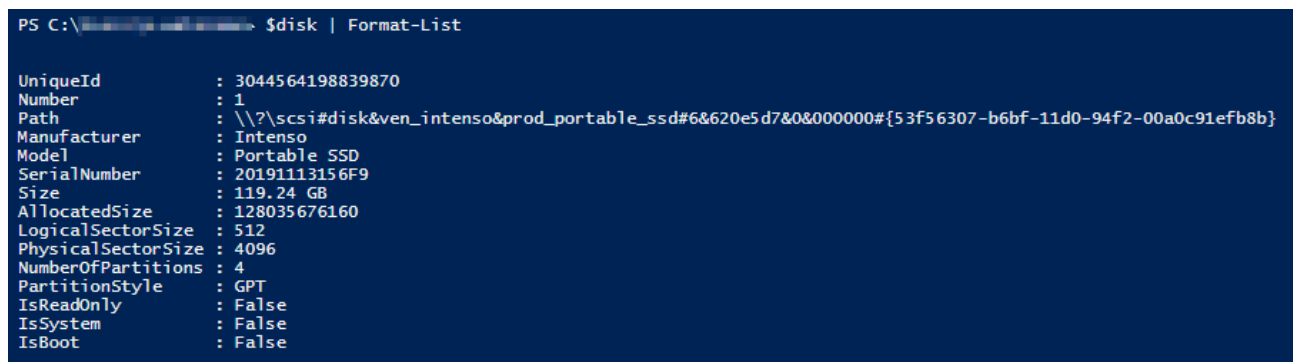
Identify the disk

Identify and select the portable disk

```
$Disk = Get-Disk | Where-Object {$_.Path -match "portable_ssd" -and $_.Size -gt 40Gb -and -not $_.IsBoot }
```

PowerShell

The first command selects all the available disks, where a disk has “portable_ssd” in the path (or name), the size is larger than 20GB and is not already a bootable device, put the result in the variable “\$disk”. In my case this results in the following output.



```
PS C:\> $disk | Format-List
UniqueId           : 3044564198839870
Number             : 1
Path               : \\?\scsi#disk&ven_intenso&prod_portable_ssd#6&620e5d7&0&000000#{53f56307-b6bf-11d0-94f2-00a0c91efb8b}
Manufacturer       : Intenso
Model              : Portable SSD
SerialNumber        : 20191113156F9
Size               : 119.24 GB
AllocatedSize       : 128035676160
LogicalSectorSize   : 512
PhysicalSectorSize  : 4096
NumberOfPartitions  : 4
PartitionStyle      : GPT
IsReadOnly          : False
IsSystem            : False
IsBoot              : False
```

Clean the disk

Now for the real scary part, we're going to clean the entire disk with the “clear-disk” cmdlet. The disk id that we will be providing with the \$Disk[0] parameter means that we select the first disk and instruct it to remove everything from the disk, including partition tables etc.

Clear the disk

```
Clear-Disk -InputObject $Disk[0] -RemoveData
```

PowerShell

Note! Disk always start at 0, partitions or volumes start at 1.

Note! You can suppress the warning message by including “-Confirm:\$false” at the end of the previous command-line.

Initializing the disk

Initializing the disk means something around the lines of defining it for a specific use. As seen in the command-line below we define a partition style of “GPT”, which means we can

use it for booting of a UEFI system. An other option would be to initialize it as a MBR (master boot record), which would be used on older systems. This guide assumes you're using this on systems with UEFI.

Initialize the disk

```
Initialize-Disk -InputObject $Disk[0] -PartitionStyle GPT
```

PowerShell

Note! If you get the message: "The disk has not been initialized.", you're already presented with a raw disk and can continue to the next step.

Partitions

First up we need a system partition, aka the partition where our boot files will reside. This is normally a partition that's not made available by means of a drive letter, but hidden from the user. As seen in the command-line, the size is just a 100 Megabytes, as it will only contains a few files. The result of creating the partition is stored in the variable "\$SystemPartition". We're going to use this variable in the next few steps.

System Partition

```
$SystemPartition = New-Partition -InputObject $Disk[0] -Size (100MB)
```

PowerShell

Note! Use "\$SystemPartition | Format-List" to get insights into the created partition.

Format the partition

```
Format-Volume -NewFileSystemLabel "System" -FileSystem FAT32 -Partition  
$SystemPartition
```

PowerShell

Here we format the partition with the FAT32 filesystem layout. This specifically needs to be FAT32 not NTFS as the latter is not part of the GPT standard. The name (Label) given to the partition is "System", but this could be anything, just use something to easily identify it.

```
PS C:\Windows\system32> Format-Volume -NewFileSystemLabel "System" -FileSystem FAT32 -Partition $SystemPartition
```

DriveLetter	FriendlyName	FileSystemType	DriveType	HealthStatus	OperationalStatus	SizeRemaining	Size
	SYSTEM	FAT32	Fixed	Healthy	OK	96 MB	96 MB

Assign the GPT Type

```
Set-Partition -InputObject $SystemPartition -GptType "{c12a7328-f81f-11d2-ba4b-  
00a0c93ec93b}"
```

PowerShell

I can understand that this is a bit confusing, let me explain. The quick rundown is that every partition on a GPT disk needs to be identified by it purpose and how it's handled by

the UEFI framework and even the operating system on that partition, even if they are “just” boot files. The firmware needs to know where to look for an initial boot of a potential operating system. Hence the ID's. A few of the common you could encounter on a Windows systems are:

GUID	Purpose
{C12A7328-F81F-11D2-BA4B-00A0C93EC93B}	EFI System Partition
{E3C9E316-0B5C-4DB8-817D-F92DF00215AE}	Microsoft Reserved Partition (MSR)
{EBD0A0A2-B9E5-4433-87C0-68B6B72699C7}	Basic data partition
{5808C8AA-7E8F-42E0-85D2-E1E90434CFB3}	Logical Disk Manager (LDM) metadata partition
{AF9B60A0-1431-4F62-BC68-3311714A69AD}	Logical Disk Manager data partition
{DE94BBA4-06D1-4D40-A16A-BFD50179D6AC}	Windows Recovery Environment
{E75CAF8F-F680-4CEE-AFA3-B001E56EFC2D}	Storage Spaces partition
{558D43C5-A1AC-43C0-AAC8-D1472B2923D1}	Storage Replica partition

source: [GUID Partition Table – Wikipedia](#)

Assign the driveletter

```
Set-Partition -InputObject $SystemPartition -NewDriveLetter "S"
```

PowerShell

Easy, right? We will still need to refer to the disk at a certain point to hold the boot files, hence we need to assign a drive letter for the time being, don't worry, you won't be seeing the drive letter once you have booted the system.

Note! You will not be seeing this drive letter in Windows Explorer as the GUID we applied implies that it should not be displayed. It's still available in the PowerShell session though.

Next up is the partition that will hold all the Windows Operating System files, this is something we should all be familiar with by now. Basically the steps are exactly the same as with the creation of the system partition, with a few differences in the details.

Windows Partition

```
$OSPartition = New-Partition -InputObject $Disk[0] -UseMaximumSize
Format-Volume -NewFileSystemLabel "Windows" -FileSystem NTFS -Partition
$OSPartition
Set-Partition -InputObject $OSPartition -GptType "{EBD0A0A2-B9E5-4433-87C0-
68B6B72699C7}"
Set-Partition -InputObject $OSPartition -NewDriveLetter "W"
```

PowerShell

The Windows system partition is created with the remaining maximum capacity (UseMaximumSize) and the result is stored in the variable "\$OSPartition". The partition is formatted with the NTFS filesystem layout. The partition is identified as a "basic data partition" by applying the GUID identifier " {EBD0A0A2-B9E5-4433-87C0-68B6B72699C7}". The last step assigns the "W" drive letter.

PartitionNumber	DriveLetter	Offset	Size	Type
1		17408	15.98 MB	Reserved
2	S	16777216	100 MB	System
3	W	121634816	119.89 GB	Basic

Note! As can be seen in the image, a magical partition appears in the output. This is a MSR (Microsoft Reserved) partition. This is a proprietary Microsoft requirement for the usage of GPT disks and partition layout. Every GPT disk will have this partition when created with Microsoft tools.

Applying Windows

The next step involves applying the Windows image to the "W:\\" partition we created. You will need to obtain a Windows 10/11 ISO file and mount it to the system first to access the containing image. If you don't have one available, there is evaluation media available for you to experiment with at the [Microsoft Evaluation Center](#).

Once downloaded, mount the file by double clicking on it or use this PowerShell command.

Applying Windows

```
Mount-DiskImage -ImagePath <path to iso>
```

PowerShell

Tip! If you want to list the assigned drive letter, use this PowerShell command as an example.

Detecting the CD Drive letter

```
(Get-Volume | where DriveType -EQ CD-ROM).DriveLetter
```

PowerShell

A Windows Image (WIM) file can contain more than one installation. So we need to know upfront what we need to refer to when we want to apply the image to the partition. The identifier is also known as an index. Use this PowerShell Command to list the available indexes in the WIM file.

List the images

```
Get-WindowsImage -ImagePath <path to wim> | select ImageName, ImageIndex
```

PowerShell

```
PS C:\Windows\system32> Get-WindowsImage -ImagePath D:\sources\install.wim | select ImageName, ImageIndex
```

ImageName	ImageIndex
Windows 11 Education	1
Windows 11 Education N	2
Windows 11 Enterprise	3
Windows 11 Enterprise N	4
Windows 11 Pro	5
Windows 11 Pro N	6
Windows 11 Pro Education	7
Windows 11 Pro Education N	8
Windows 11 Pro for Workstations	9
Windows 11 Pro N for Workstations	10

In the image above we can see that Windows 11 Enterprise has index number 3, let's use that. The command for applying this specific image to our partition "W:\\" will be as following:

Apply the image

```
Expand-WindowsImage -ImagePath "D:\sources\install.wim" -Index 3 -ApplyPath "w:\\"
```

PowerShell

Just give it time, it will take a few minutes to complete. Once it's done, you can see a regular Windows installation being applied on the "W:\\" partition.

No auto mount

As the disk will contain critical boot files and eventually confidential files, it's best to make sure that the disk will not be visible per default on another system. What I mean with that is that when you insert the disk in already a booted system, it will usually present you with a dialog, asking what you want to do with the disk. There are a couple of ways you can prevent that from happening.

Windows partition only

```
Set-Partition -InputObject $OSPartition -NoDefaultDriveLetter $TRUE
```

PowerShell

This will still mount the disk, but the Windows partition will not be assigned a drive letter. Obviously anyone with administrator rights will be able to assign a drive letter.

SAN Policy

Another way to instruct the Operating System which drive letter the USB will get (or not), is by means of applying a SAN Policy. In essence this is a way to force mounting or denying mounting available partitions. There are 3 values available:

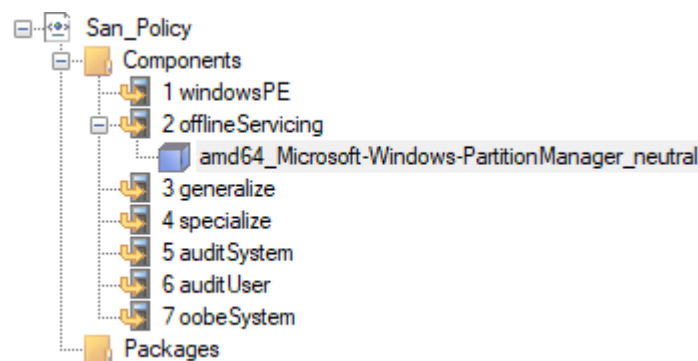
Value	Description
1	Mounts all available storage devices. This is the default value.
2	Mounts all storage devices except those on a shared bus. Examples of shared buses are: SCSI, iSCSI, Fiber, and SAS.
3	Does not mount storage devices.

Source: [SanPolicy | Microsoft Learn](#)

Applying the policy involves two steps, creating the SAN Policy in XML and applying it with DISM.

Create the SAN Policy

The policy can easily be created in the “Windows System Image Manager (WSIM)”, part of the “Assessment and Deployment Toolkit” (ADK). Open up the Windows image in the WSIM, create a new unattended file and add the “amd64_Microsoft-Windows-PartitionManager_neutral” component to the sequence “OfflineServicing”, apply the SanPolicy of 3 and save the file.



Next up is applying the SAN Policy, we’re going to use DISM for that. You don’t need the file that’s included with the ADK , the version included with Windows will do perfectly.

Apply the policy

```
dism.exe /Image:W:\ /Apply-Unattend:W:\san_policy.xml
```

BAT (Batchfile)

In the example above, the “san_policy.xml” is also stored on the “W:\” partition, but it can be stored anywhere.

SAN Policy

```
<?xml version="1.0" encoding="utf-8"?>
<unattend xmlns="urn:schemas-microsoft-com:unattend">
  <settings pass="offlineServicing">
```

```

        <component language="neutral" name="Microsoft-Windows-PartitionManager"
processorArchitecture="amd64" publicKeyToken="31bf3856ad364e35"
versionScope="nonSxS" xmlns:wcm="http://schemas.microsoft.com/WMIconfig/2002/State"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
        <SanPolicy>3</SanPolicy>
    </component>
</settings>
<cpi:offlineImage
cpi:source="wim://deploy/deploymentshare$/operating%20systems/windows%2011%20busine
ss%20editions%20-%20build%2022h2/sources/boot.wim#Microsoft Windows Setup (amd64)"
xmlns:cpi="urn:schemas-microsoft-com:cpi" />
</unattend>

```

XML

Note! A SAN Policy can only be applied after the Windows System has been copied onto the partition.

Windows Recovery

Per default the Windows recovery environment, a specialized version of Windows PE with additional tools to service your machine in case of an incident will need to be available within your partition layout. Since we didn't make one during the partitioning we need to tell the installer to skip it. This again can be achieved with an unattended file. Use the same procedure as in the previous paragraph and add the component "amd64_Microsoft-Windows-WinRE-RecoveryAgent_neutral" to the "oobeSystem" sequence. Set the value "UninstallWindowsRE" to true. Save the file as "unattended.xml".

As of this point, anything that you want to automate can be put in this file, just use the WSIM to guide you in the process. Once you're satisfied with the result copy the file to "W:\Windows\System32\Sysprep\unattended.xml".

Unattended file

```

<?xml version="1.0" encoding="utf-8"?>
<unattend xmlns="urn:schemas-microsoft-com:unattend">
    <settings pass="oobeSystem">
        <component name="Microsoft-Windows-WinRE-RecoveryAgent"
processorArchitecture="amd64" publicKeyToken="31bf3856ad364e35" language="neutral"
versionScope="nonSxS" xmlns:wcm="http://schemas.microsoft.com/WMIconfig/2002/State"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
            <UninstallWindowsRE>true</UninstallWindowsRE>
        </component>
    </settings>
    <cpi:offlineImage
cpi:source="wim://deploy/deploymentshare$/operating%20systems/windows%2011%20busine
ss%20editions%20-%20build%2022h2/sources/boot.wim#Microsoft Windows Setup (amd64)"
xmlns:cpi="urn:schemas-microsoft-com:cpi" />
</unattend>

```

Boot Files

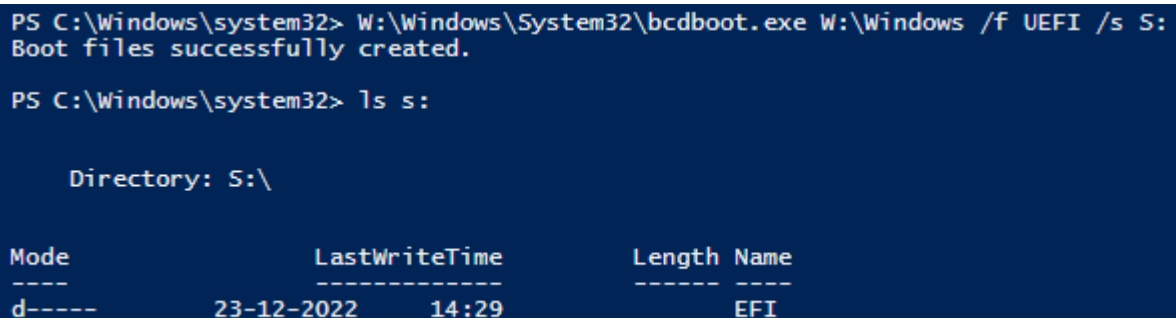
One last step that needs to be taken is creating the Boot Critical Database (BCD) and copying the additional boot files to the partition we defined as system and given the drive letter “S:\”. Luckily we can easily accomplish that by using the “bcdboot” utility. As a tip, always use the “bcdboot” from the system you have applied on the USB disk. In our case this would mean using the tool in “W:\Windows\System32”. The entire command-line would look like this.

Apply the boot files

```
W:\Windows\System32\bcdboot.exe W:\Windows /f UEFI /s S:
```

BAT (Batchfile)

Let me explain this command-line. The first parameter “W:\Windows” tells the bcdboot program which Windows installation should be referred to while creating the BCD store. “/f UEFI” tells the store what kind of system you’re booting from, since we want to use it exclusively from a UEFI system, we use the UEFI parameter. The latter part of the command-line, “/s S:” tells the system where the boot files will need to be stored.



```
PS C:\Windows\system32> W:\Windows\System32\bcdboot.exe W:\Windows /f UEFI /s S:
Boot files successfully created.

PS C:\Windows\system32> ls s:

    Directory: S:\

Mode                LastWriteTime         Length Name
----                -
d-----          23-12-2022    14:29             EFI
```

Tip! If you want to take a look at the created BCD store on the USB disk, use the bcdedit tool to enumerate the store.

Enumerate the store

```
bcdedit /store S:\EFI\Microsoft\Boot\BCD
```

BAT (Batchfile)

```

PS C:\Windows\system32> bcdedit /store S:\EFI\Microsoft\Boot\BCD

Windows Boot Manager
-----
identifier                {bootmgr}
device                    partition=S:
path                      \EFI\Microsoft\Boot\bootmgfw.efi
description                Windows Boot Manager
locale                    en-us
inherit                    {globalsettings}
default                    {default}
resumeobject                {e0fa97e3-82c5-11ed-a249-ae8161fb3770}
displayorder                {default}
toolsdisplayorder          {memdiag}
timeout                    30

Windows Boot Loader
-----
identifier                {default}
device                    partition=W:
path                      \Windows\system32\winload.efi
description                Windows 11
locale                    en-us
inherit                    {bootloadersettings}
isolatedcontext            Yes
allowedinmemorysettings    0x15000075
osdevice                    partition=W:
systemroot                \Windows
resumeobject                {e0fa97e3-82c5-11ed-a249-ae8161fb3770}
nx                        OptIn
bootmenupolicy              Standard

```

Booting the system

This is the fun part, booting the system and see if we have achieved the desired result. We can actually accomplish this multiple ways. By directly accessing the firmware boot menu and selecting our disk or we can edit the “Boot Critical Database” (BCD) and use the Windows boot manager to determine what partition we want to load. In case of the hardware option it’s really depended on your manufacturer. On my HP laptop for example I have to press F9 when turning on my machine, select the boot menu and disk, and off we go. This boot method is very inconsistent across all hardware though. I personally prefer to use the Windows boot manager for booting a secondary Windows installation.

As you’ve seen in the previous paragraph, the tool “bcdedit” can be used to view the boot store, but luckily for us we can also edit the store directly. Open up an elevated command prompt and list the content of your current boot store.

List the boot entries

```
bcdedit /enum /v
```

BAT (Batchfile)

```

C:\Windows\System32>bcdedit /enum /v

Windows Boot Manager
-----
identifier                {9dea862c-5cdd-4e70-acc1-f32b344d4795}
device                    partition=\Device\HarddiskVolume1
path                      \EFI\Microsoft\Boot\bootmgfw.efi
description               Windows Boot Manager
locale                    en-US
inherit                   {7ea2e1ac-2e61-4728-aaa3-896d9d0a9f0e}
default                   {f5db9d5a-7dab-11ed-a127-b061867e11ae}
resumeobject              {f5db9d59-7dab-11ed-a127-b061867e11ae}
displayorder              {f5db9d5a-7dab-11ed-a127-b061867e11ae}
toolsdisplayorder         {b2721d73-1db4-4c62-bf78-c548a880142d}
timeout                   30

Windows Boot Loader
-----
identifier                {f5db9d5a-7dab-11ed-a127-b061867e11ae}
device                    partition=C:
path                      \Windows\system32\winload.efi
description               Windows 11
locale                    en-US
inherit                   {6efb52bf-1766-41db-a6b3-0ee5eff72bd7}
recoverysequence          {f5db9d5b-7dab-11ed-a127-b061867e11ae}
displaymessageoverride    Recovery
recoveryenabled            Yes
isolatedcontext            Yes
allowedinmemorysettings  0x15000075
osdevice                  partition=C:
systemroot                \Windows
resumeobject              {f5db9d59-7dab-11ed-a127-b061867e11ae}
nx                         OptIn
bootmenupolicy             Standard

```

In the image above, I've highlighted the identifier part of the Windows Boot Loader, which is the current Windows version we're using to prep the USB drive in this post. We can actually use this boot loader as a starting point to create our own entry by simply using the `bcdedit copy` command.

Copy the boot entry

```
bcdedit /copy {f5db9d5a-7dab-11ed-a127-b061867e11ae} /d "Windows To Go"
```

BAT (Batchfile)

Tip! Instead of using the GUID, you can refer to the current boot entry by replacing the GUID with `{current}`.

Pay special attention to the newly created GUID, as this will be our new reference point.

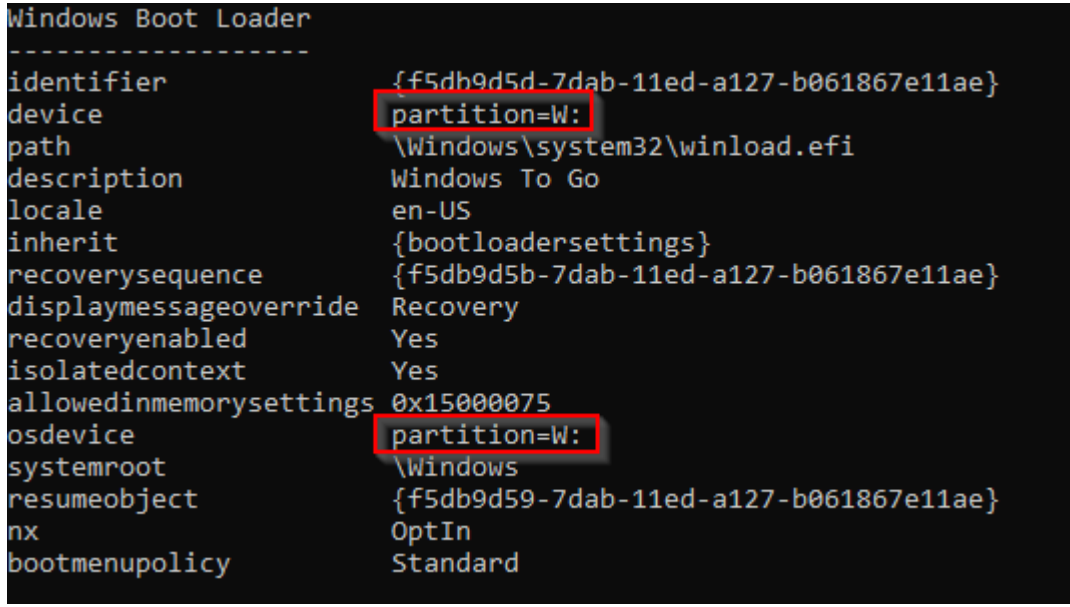
You need to change two crucial settings in the boot loader entry. These settings tell the Windows Boot Manager to look for the Windows installation on the USB disk during initialization. Type the following commands with the appropriate GUID to alter the device and `osdevice` settings of the new boot entry.

Device and OSdevice

```
bcdedit /set {f5db9d5d-7dab-11ed-a127-b061867e11ae} device partition=W:  
bcdedit /set {f5db9d5d-7dab-11ed-a127-b061867e11ae} osdevice partition=W:
```

BAT (Batchfile)

The end result will look like this:



```
Windows Boot Loader  
-----  
identifier                {f5db9d5d-7dab-11ed-a127-b061867e11ae}  
device                    partition=W:  
path                      \Windows\system32\winload.efi  
description               Windows To Go  
locale                   en-US  
inherit                   {bootloadersettings}  
recoverysequence          {f5db9d5b-7dab-11ed-a127-b061867e11ae}  
displaymessageoverride    Recovery  
recoveryenabled           Yes  
isolatedcontext           Yes  
allowedinmemorysettings  0x15000075  
osdevice                  partition=W:  
systemroot               \Windows  
resumeobject              {f5db9d59-7dab-11ed-a127-b061867e11ae}  
nx                        OptIn  
bootmenupolicy            Standard
```

Hint! If, at a later stage you want to remove the boot loader entry, just use the delete option within bcdedit.

Delete the boot entry

```
bcdedit /delete {f5db9d5d-7dab-11ed-a127-b061867e11ae}
```

BAT (Batchfile)

Safeguard the USB Boot disk

Once the system has been installed and you're happy with the result, (clap your hands) reboot back to your regular Windows installation and make a full sector based copy of your USB disk with DISM. First we need to find a reference to your disk. Open up an elevated PowerShell and use the following command:

List the disk

```
Get-Disk | Where-Object {$_.Path -match "portable_ssd"} | select number
```

PowerShell

Note the number for the next command.

Next we're going to use the full disk imaging capability of DISM, no need for a third party application for that. In the same elevated command prompt, use the following command-line and adjust to your needs.

Cope the disk

```
DISM.exe /Capture-ffu /ImageFile:c:\temp\wtg.ffu /CaptureDrive:\\.\PhysicalDrive1  
/Name:"Windows to Go" /Compress:default
```

PowerShell

Hint! Replace the number after the word “PhysicalDrive” with the number from the first PowerShell command we’ve used in the beginning of this paragraph.

And that’s it! I hope this information is useful to you and please let me know if you have any questions. The files I’ve used during this blog post are included in the link below.

Downloads

[Windows to go files](#)

Reference

[DISM Image Management Command-Line Options | Microsoft Learn](#)